TASK 06 – ORGANISE THE LOGIC INTO FUNCTIONS

Assignment: Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative
Bloom's Level: K3 – Apply
Marks: 4

1. Aim

To divide the milk payment calculation into reusable Python functions and understand parameters, return values, local and global variables, and docstrings.

2. Functions Required

A function is a reusable block of code that performs a particular task.

The milk payment calculator can be divided into three main functions:

Function	Parameters	Return value
rate_per_litre()	Fat and SNF	Milk rate per litre
payout()	Litres, fat and SNF	Payment amount
ten_day_total()	List of farmer records	Total payment for all records

This makes the program easier to understand, test and maintain.

3. Parameters and Return Values

Parameters are the values passed to a function.

A return statement sends the calculated result back to the place where the function was called.

Example:

```
def add(a, b):
    return a + b


result = add(10, 20)
print(result)
```

Output:

30

4. Difference Between return and print

- return sends a result back to the calling program so it can be reused.
- print() only displays a value on the screen.

Example:

```
def calculate():
    return 100


amount = calculate()
print(amount)
```

The returned value can be stored and used in another calculation.

If the function only uses print(100), the displayed value cannot be directly reused as its return value.

5. Local and Global Variables

A global variable is declared outside a function and can be accessed inside functions.

A local variable is created inside a function and is normally available only within that function.

Example:

```
BASE_RATE = 30.00


def show_rate():
    extra = 2.00
    print(BASE_RATE + extra)


show_rate()
```

Here, BASE_RATE is global and extra is local.

Rate constants should be read inside calculation functions but should not be changed inside them. This keeps the rate card consistent.

6. Docstrings

A docstring is a short description written inside a function to explain its purpose.

Example:

```
def payout(litres, fat, snf):
    """Calculate and return the farmer's payment."""
    return litres
```

Docstrings make functions easier to understand and maintain.

7. Alternative Concept and Complexity Analysis

Single long script: The entire calculation can be written in one continuous block. However, repeated calculations may require copying the same statements. This makes the program harder to maintain and increases the chance of errors.

Using functions: The calculation is written once and called whenever needed. This reduces repeated code and makes testing easier.

Using lambda expressions: A lambda is a small anonymous function. It is suitable for simple expressions, but the normal def function is clearer for a calculation involving multiple steps and documentation.

Complexity analysis:

- Calculating one farmer's payment takes O(1) time because it uses a fixed number of operations.
- Calculating payments for n farmer records takes O(n) time.
- The total calculation uses O(1) extra space if only a running total is stored, excluding the input records.
- Function calls have a small overhead, but they do not change the overall O(n) time complexity.

8. Result

The milk payment calculation was divided into reusable functions. Parameters, return values, local and global variables, and docstrings were demonstrated. Functions make the program easier to reuse, test and maintain.